

Concurrency & Distributed Systems

Week 3: Synchronization, Memory Semantics & Deadlocks

Day 1 — Part 2: What synchronized Guarantees and How It Eliminates Races

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

A Simple Analogy Before We Begin

Think of a single-key room

A synchronized block is like a room with one key:

- only one person can enter at a time
- everyone else must wait outside
- when the person leaves, the room state is now up to date for the next person

Why this analogy helps

It captures both ideas: **exclusive access** and **seeing the latest state**.

Block 3 — What synchronized Really Guarantees

This is the conceptual pillar

synchronized does **two** things:

- it provides **mutual exclusion**
- it provides **visibility guarantees**

Key message

If students understand only “one thread at a time,” they understand only half of synchronization.

Guarantee 1 — Mutual Exclusion

Definition

Mutual exclusion means that no two threads execute the same protected critical section at the same time.

- One thread acquires the monitor
- Other threads trying to enter become **BLOCKED**
- The protected code runs in an exclusive region

Why it matters

This prevents unsafe interleaving of read–modify–write operations such as `count++`.

Guarantee 2 — Visibility

Definition

Visibility means that writes made by one thread become observable to another thread in a well-defined way.

- When a thread exits a synchronized block, its writes are committed so other threads can observe them
- When another thread later enters a synchronized block guarded by the same monitor, it sees fresh values

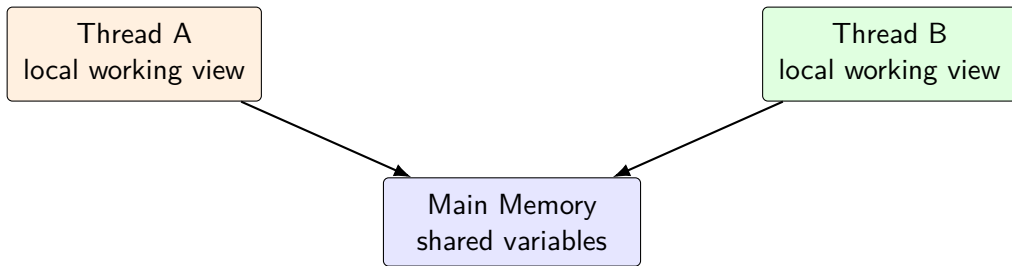
Important

synchronized is not only about “keeping threads apart.” It also governs what values threads are allowed to see.

A Simplified Java Memory Model View

Mental model

Each thread may work with its own temporary view of memory, while shared variables conceptually live in main memory.



Why synchronization matters

Without proper synchronization, one thread may keep seeing an old value even after another thread has updated it.

Formal Java Memory Model Rule

Happens-before rule

Unlock of monitor M happens-before subsequent lock of monitor M .

- Thread A exits a synchronized block on monitor M
- Thread B later enters a synchronized block on the same monitor M
- Thread B is guaranteed to observe the writes made by Thread A before the unlock

Interpretation

This is not an implementation trick. It is a formal guarantee of the Java Memory Model.

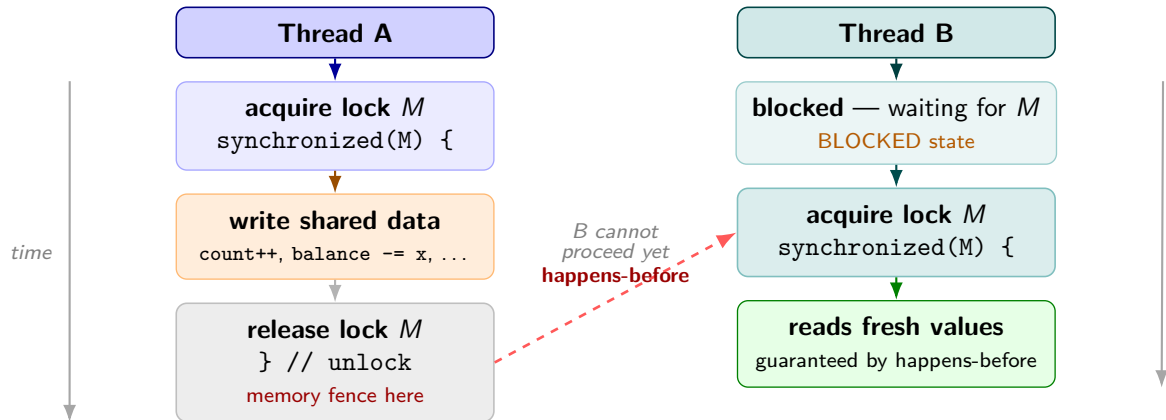
Mutual Exclusion vs Visibility

Concept	Meaning
Mutual Exclusion	Only one thread inside the critical section
Visibility	Writes become visible to other threads

Students must distinguish them

A program may avoid overlap yet still have stale reads if visibility is not guaranteed.

A Simple Timeline of Visibility



Core idea

The unlock is not just a release of the lock. It is a **memory-ordering event** — all of Thread A's writes become visible to Thread B when B subsequently acquires the same monitor.

Common Mistake

Wrong mental model

“synchronized only prevents two threads from running together.”

Correct mental model

synchronized provides:

- exclusive entry into a critical section
- visibility of prior writes across threads

Mutual exclusion $\neq$ visibility

Check Your Understanding

Question

If two threads never execute a critical section at the same time, does that automatically mean they always see the latest shared values?

- ① Yes, always
- ② No, not unless visibility is also guaranteed
- ③ Yes, because exclusion and visibility are the same thing

Expected answer

2. Preventing overlap is not the same as guaranteeing fresh reads.

Reentrancy Note

Important property

Java intrinsic locks are **reentrant**.

- If a thread already holds a monitor, it can enter another synchronized section guarded by the same monitor
- The same thread does not block itself on the same lock

Why mention this now?

This prevents confusion later when synchronized methods call other synchronized methods on the same object.

Block 4 — Fixing the Counter Properly

Goal

We now repair the counter using a **shared lock object**.

- Before: unsynchronized increment
- After: protected critical section

Question

What exactly does the fix eliminate?

Counter State with Shared Lock

```
class Counter {  
    static int count = 0;  
    static final Object lock = new Object();  
}
```

Design choice

The counter is shared, so the lock must also be shared.

Why final?

The identity of the lock must not change.

Protected Increment

```
synchronized(Counter.lock) {  
    Counter.count++;  
}
```

- All threads compete for the same monitor
- Only one thread increments at a time
- Each increment becomes visible to the next thread that acquires the lock

Before vs After Synchronization

Before	After
Lost updates	No lost updates
Unsafe interleaving	Mutual exclusion
No visibility guarantee	Visibility guaranteed
Broken invariant	Race eliminated

Key phrase

Race elimination through synchronization

Why the Repair Works

- ① The critical section is now protected by one shared monitor
- ② Read, modify, and write occur without overlap from other threads
- ③ Unlock of the monitor makes writes visible
- ④ The next thread sees the latest value before incrementing

Result

The invariant is restored because both interleaving errors and stale observations are controlled.

Spot the Bug

```
void increment() {  
    synchronized(new Object()) {  
        count++;  
    }  
}
```

What is wrong here?

A new lock object is created every time, so threads are **not** synchronizing on the same monitor.

Lesson

Synchronization works only when threads contend for the **same lock object**.

Minimal Runnable Example

```
class Counter {  
    static int count = 0;  
    static final Object lock = new Object();  
  
    static void increment() {  
        synchronized(lock) {  
            count++;  
        }  
    }  
}
```

Exercise

Run the unsynchronized and synchronized versions side by side.

A Small Preview on Performance

Important trade-off

Correct synchronization is necessary, but locking too broadly can reduce concurrency.

- lock too little → unsafe program
- lock too much → correct but slower program

Preview

Later we will discuss lock granularity and alternatives such as `AtomicInteger`.

New case study

The bug is no longer just a counter mismatch. It is a violation of business logic.

Core issue

The check and the update must be atomic together.

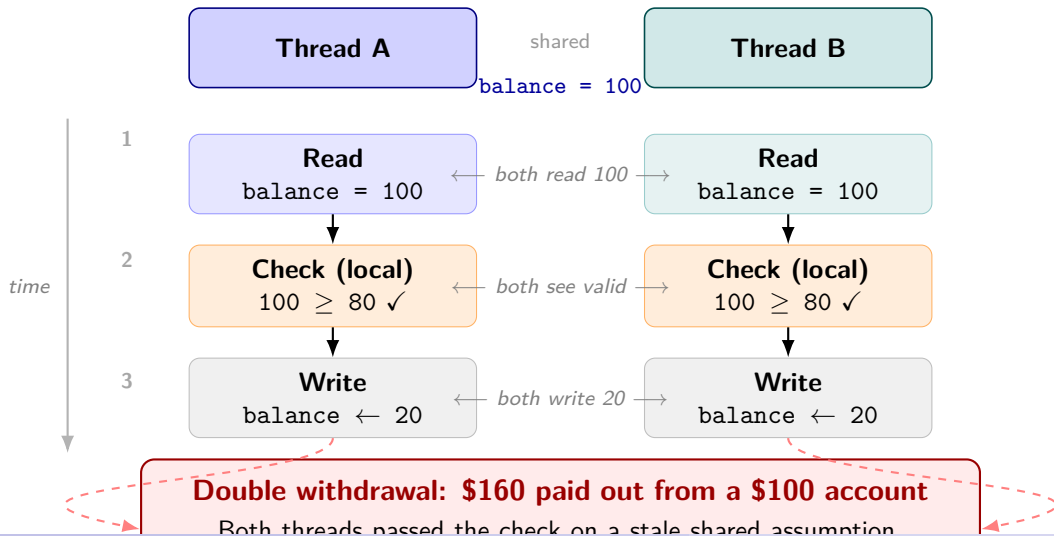
Unsafe Version

```
public void withdraw(int amount) {  
    if (balance >= amount) {  
        balance -= amount;  
    }  
}
```

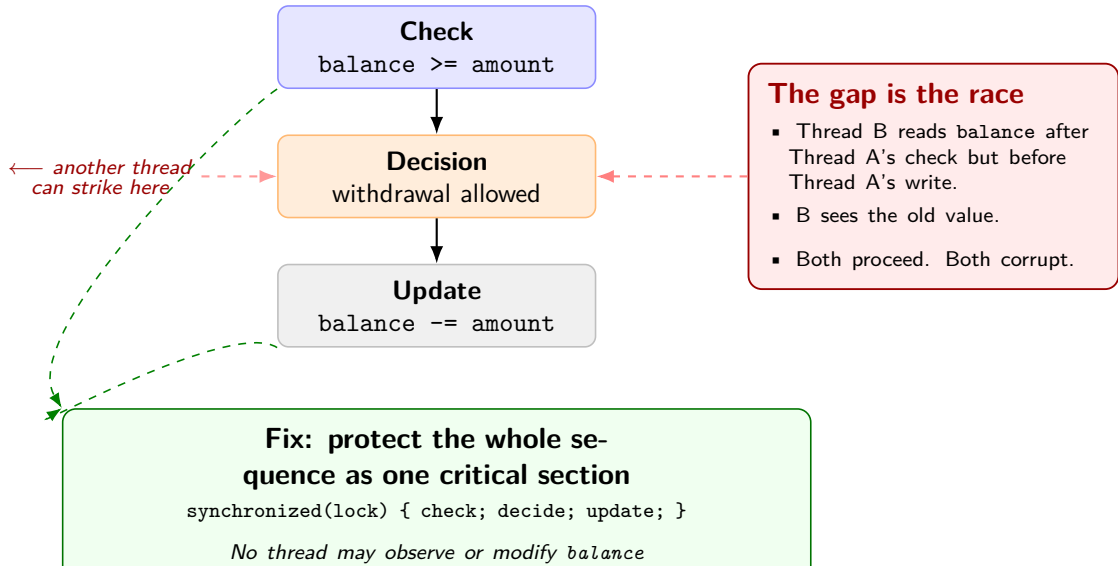
Why unsafe?

The condition check and the update are separate actions. Another thread may interleave between them.

Unsafe Interleaving Example



Why Check + Update Must Stay Together




```
public synchronized void withdraw(int amount) {  
    if (balance >= amount) {  
        balance -= amount;  
    }  
}
```

Why this works

For an instance method, `synchronized` uses the monitor of `this`.

Why Method-Level Synchronization Works

- The method is guarded by the instance monitor
- Only one thread at a time may execute `withdraw()` on the same account object
- The check and the update are now one atomic critical section
- Visibility of the updated balance is guaranteed for subsequent synchronized access

Important

This works only if threads operate on the same account instance.

Instance Lock vs Static Lock

Form	Lock used	Scope
synchronized instance method	<code>this</code>	one object instance
static synchronized method	<code>ClassName.class</code>	all instances of the class

Interpretation

Choose the lock scope based on what state is shared: shared per object, or shared across all objects.

Common Pitfalls

- Using different lock objects for the same shared data
- Locking on an object whose identity may change
- Assuming synchronized methods make every called method automatically thread-safe
- Confusing visibility with atomicity

Preview

A `volatile` variable can help with visibility, but it does **not** make compound operations like `count++` atomic.

Lab Discussion Prompts

- ① Why is the original `withdraw()` unsafe?
- ② What makes the repaired version atomic?
- ③ What object is actually used as the lock?
- ④ When would `static synchronized` be appropriate instead?

Teaching objective

Students should connect code syntax to monitor identity and shared-state scope.

Part 2 Summary

- synchronized guarantees both mutual exclusion and visibility
- Unlock of monitor M happens-before subsequent lock of M
- A shared lock can eliminate the counter race
- The bank-account bug exists because check and update were not atomic together
- Method-level synchronization uses the instance monitor

Bridge forward

Next we move from race elimination to deadlock: what happens when threads wait on each other's locks?